SYNCLAB: Centralized Workshop & Laboratory Allocation Platform

UML Class Diagram & Structural Architecture Specification

System Architecture Document | **Implementation:** C++17 / Qt Widgets / SQLite

System Architecture Overview

The SYNCLAB system implements a modular layered object-oriented architecture in C++17/Qt. It decouples GUI widget controllers from core domain rules and persistence transactions using an **Inversion of Control (IoC) / Repository Pattern**.

- **Presentation Layer:** `MainWindow` inherits from `QMainWindow` and manages screen states via `QStackedWidget`. It embeds `CustomCalendarWidget` (extending `QCalendarWidget`) to paint status badges.
- **Domain Model & Logic:** `StudentRecord` serves as an encapsulated Data Transfer Object (DTO). `AllocationRules` acts as a static business service validating booking inputs.
- **Persistence Layer:** `IDatabaseManager` defines an abstract CRUD contract, concretely realized by `SqliteDatabaseManag` using ACID transactions and cascading SQLite foreign keys.

UML Class Diagram

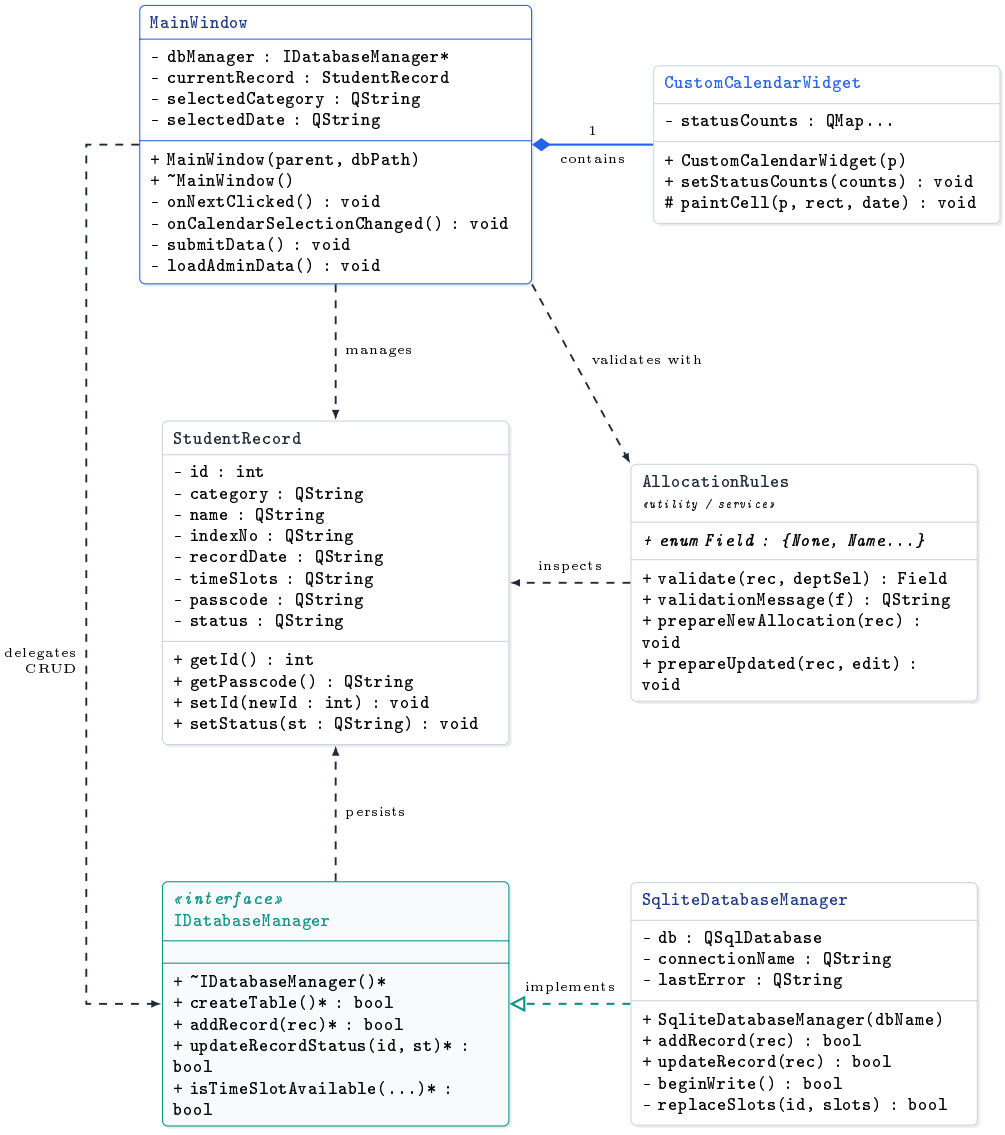


Figure 1: UML Class Diagram for SYNCLAB Structural Architecture

Class Specifications & Contracts

Presentation Layer

- `MainWindow`: Primary controller inheriting from `QMainWindow`. Holds a private interface pointer `IDatabaseManager* dbManager`. Coordinates state transitions across the 5 wizard interfaces via Qt slots (`onNextClicked`, `submitData`, `loadAdminData`).
- `CustomCalendarWidget`: Subclasses `QCalendarWidget`. Overrides `paintCell(QPainter*, QRect, QDate)` to render Approved, Pending, and Rejected badge counts directly onto calendar days.

Domain & Business Logic Layer

- `StudentRecord`: Encapsulated entity modeling 15 fields including identifiers (`id`, `passcode`), student credentials (`indexNo`, `name`, `department`, `email`), allocation metadata (`category`, `recordDate`, `timeSlots`, `supervisorName`, `projectName`), and audit records (`status`, `adminRemarks`, `createdAt`).
- `AllocationRules`: Static domain service enforcing validation rules (non-empty entries, regex email formats, valid slot sets) and state initialization without coupling to the GUI.

Data Access Layer

- `IDatabaseManager`: Pure abstract repository interface defining transactions, slot conflict checks (`isTimeSlotAvailable`), and status modifications.
- `SqliteDatabaseManager`: Concrete driver using `QSqlDatabase`. Executes foreign-key-enabled schema setups and multi-row slot updates within atomic SQLite transactions.

Relationship & Multiplicity Mapping

The structural coupling, association types, and multiplicity constraints between architectural components are summarized below:

Source Class	Target Class	Relationship	Multiplicity	Architectural Role / Contract
`MainWindow`	`CustomCalendarWidget`	Composition	$1 \rightarrow 1$	Embedded UI component; owned directly by parent window.
`MainWindow`	`StudentRecord`	Dependency	$1 \rightarrow 1$	Active session state holder passed across wizard views.
`MainWindow`	`AllocationRules`	Dependency	$1 \rightarrow 1$	Static business service invoked for pre-submission checks.
`MainWindow`	`IDatabaseManager`	Dependency	$1 \rightarrow 1$	Operates via repository interface pointer (IoC pattern).
`SqliteDatabaseManager`	`IDatabaseManager`	Realization	$1 \rightarrow 1$	Implements pure virtual CRUD and transaction contracts.
`AllocationRules`	`StudentRecord`	Dependency	$1 \rightarrow 1$	Inspects record fields to validate constraints.
`IDatabaseManager`	`StudentRecord`	Dependency	$1 \rightarrow *$	Serializes, queries, and mutates database records.

Table 1: System Structural Relationships and Multiplicity Summary